

Поведение алгоритмов суммирования чисел с плавающей точкой при различных типах входных данных.

[исходный код проекта](#) (Github)

1 . Введение

Числа с плавающей точкой используются очень широко как способ представлять вещественные величины. Такой способ обладает ограниченной точностью, но обеспечивает высокую скорость вычислений и широкий диапазон значений. Из-за конечной разрядности представления могут переставать выполняться базовые арифметические свойства. По этой причине разные алгоритмы могут возвращать отличающуюся сумму одного и того же набора данных. Этот эффект особенно заметен в областях, где обрабатываются большие массивы данных, например, научные исследования, статистика или численное моделирование. Интерес к данной теме возник из наблюдения, что арифметика чисел с плавающей точкой не обладает свойством ассоциативности. Желание разобраться, почему так происходит, и какие существуют методы решения данной проблемы привело меня к данной теме исследования. В данной работе проводится сравнение нескольких алгоритмов суммирования чисел с плавающей точкой на различных типах входных данных. Оценка производится по критериям точности и производительности с использованием специально разработанных наборов данных.

2 . Цели и задачи

Цель: провести сравнительный анализ некоторых алгоритмов суммирования по критериям точности и времени выполнения. Узнать, какие характеристики входных данных влияют на точность и производительность алгоритма.

Задачи:

- Изучить существующие алгоритмы суммирования чисел с плавающей точкой
- Выбрать алгоритмы и реализовать их.
- Реализовать способ генерации воспроизводимых тестовых данных с различными характеристиками.
- Протестировать алгоритмы и визуализировать полученные данные.
- Определить область применения для каждого алгоритма.

3 . Методы и алгоритмы

3 . 1 Методика исследования:

Тестировалось три алгоритма суммирования - {наивное суммирование; алгоритм попарного суммирования; алгоритм Кэхэна}. За эталонную реализацию алгоритма суммирования была взята `math.fsum()` из библиотеки `math` в `python`. Для каждого алгоритма генерировались 5 вариантов датасетов {случайные вещественные числа, принадлежащие диапазону $[-1;1]$; сокращающиеся пары; сокращающиеся пары с шумом; числа с сильно различающейся экспонентой; первые n членов произвольной гармонической последовательности}. Для каждой пары (алгоритм суммирования; датасет) проводилось 4 теста {неотсортированный массив; отсортированный по неубыванию; отсортированный по невозрастанию; отсортированный по модулю по неубыванию}. Тесты проводились для разных размеров массива (n), сиды для каждого такого теста не пересекались и вычислялись по формуле `seed = n + trial`, где n - `trial` - индекс испытания. `trial` $\in [0;49]$ и $\in \mathbb{N}$. На графиках представлено среднее время выполнения, средняя относительная и абсолютная ошибки.

Время выполнения вычислялось с помощью `perf_counter()` из библиотеки `time`.

Абсолютная ошибка вычислялась по формуле: $|S_{ref} - S|$, где S_{ref} - референсная сумма, а S - результат работы тестируемого алгоритма суммирования.

Относительная ошибка вычислялась по формуле: $\frac{|S_{ref} - S|}{|S_{ref}|}$ только в тестах, где $S_{ref} \neq 0$

3.2 Устройство директории с [исходным кодом проекта](#):

В директории `./csv` находятся сырые данные тестирования для каждого набора (алгоритм суммирования, вариант датасета, вариант теста) в формате {Название алгоритма}_{Вариант датасета}_{Вариант теста}.csv .

В директории `./images` находятся графики для каждого теста. Для удобства, они отсортированы по разным директориям. Директория `./images/v` содержит графики, на которых нанесен лишь один алгоритм и вариант теста. В директории `./images/v3` находятся графики, позволяющие сравнить алгоритмы между собой. В директории `./images/v4` находятся графики, позволяющие сравнить варианты теста между собой на одном алгоритме. Графиков получилось довольно много, поэтому в некоторых пунктах я описал общие сведения из них и выбрал наиболее показательные.

В файле `algorithms.py` описаны алгоритмы суммирования.

В файле `dataset_generators.py` описаны генераторы данных.

В файле `plots.py` описана функция генерации изображений с графиками.

В файле `tests.py` описаны функции тестирования алгоритмов.

Файл `main.py` служит основным файлом, из которого вызываются все функции.

В файле `config.py` находятся настройки:

- `seed_count` - количество различных сидов на каждый тест.

- `sizes` - список с тестируемыми размерами массивов. Должны располагаться в порядке возрастания. Примечание: если разница между двумя размерами массивов $< seed_count$, то случится пересечение сидов из-за методики его вычисления.

- `volume_randoms` - найстрока диапазона значений для генераторов данных с сокращающимися парами (с шумом и без).

- `volume_eps` - диапазон, в котором будет генерироваться шум.

- `exp_volume` — диапазон, в котором генерируются экспоненты в алгоритме генерации чисел разного порядка.

3.3 Реализации алгоритмов на python:

Алгоритм Кэхэна:

```
def kahan_sum(arr):  
    summ = np.float64(0)  
    p = np.float64(0)  
    for i in range(len(arr)):  
        k = arr[i] - p  
        l = summ + k  
        p = (l - summ) - k  
        summ = l  
    return summ
```

наивный алгоритм:

```
def naive_sum(arr):  
    summ = np.float64(0)  
    for i in arr:  
        summ += i  
    return summ
```

Алгоритм попарного сложения:

```
def pairwise_sum(arr):  
    if len(arr) == 0: return np.float64(0)  
    elif len(arr) == 1: return np.float64(arr[0])  
    else: return pairwise_sum(arr[:len(arr)//2]) +  
pairwise_sum(arr[len(arr)//2:])
```

3.4 Используемое ПО:

Язык программирования Python 3.14.4.

Библиотека NumPy 2.4.4 для работы с массивами, генерации случайных данных и сохранения результатов экспериментов.

Библиотека matplotlib 3.10.9 для генерации графиков.

Встроенные в Python библиотеки math и time.

3.5 Иные подробности:

Тесты производились на процессоре Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz

Операционная система: Artix Linux (openRC)

Версия ядра: Linux 7.0.3-artix1-2

Эксперименты выполнялись последовательно в однопоточном режиме.

4 . Результаты тестов

4 . 1 Абсолютная ошибка

Наивное суммирование показало наибольшую абсолютную ошибку при всех типах входных данных, кроме тех, которые представляют собой сокращающиеся (или почти сокращающиеся) пары, отсортированные по модулю (см. Графики 1, 2, 3, 4 в приложении 4.1). Так происходит, потому что наивное суммирование копипастит все погрешности, пока суммирует числа. Та погрешность, которая произошла при первых вычислениях участвует во всех последующих операциях и накапливается. В случае с сокращающимися (или почти) парами погрешности либо равны нулю, либо очень малы.

Наилучший результат при всех типах входных данных алгоритм Кэхэна. Такая точность достигается тем, что помимо суммы алгоритм хранит и вычисляет дополнительную компенсационную величину, позволяющую учитывать часть ошибок округления, возникших на предыдущих шагах суммирования. В случае со случайными вещественными числами $\in [-1;1]$ алгоритм Кэхэна не показал ошибки относительно эталонной суммы.

При этом, все алгоритмы показали себя очень плохо в случае несортированными массивами чисел, у которых сильно отличается экспонента. Минимальная абсолютная ошибка была у алгоритма Кэхэна - 120000, а у наивного суммирования целых $4 \cdot 10^9$ (см. графики 5, 6 в приложении).

Приложение 4.1:

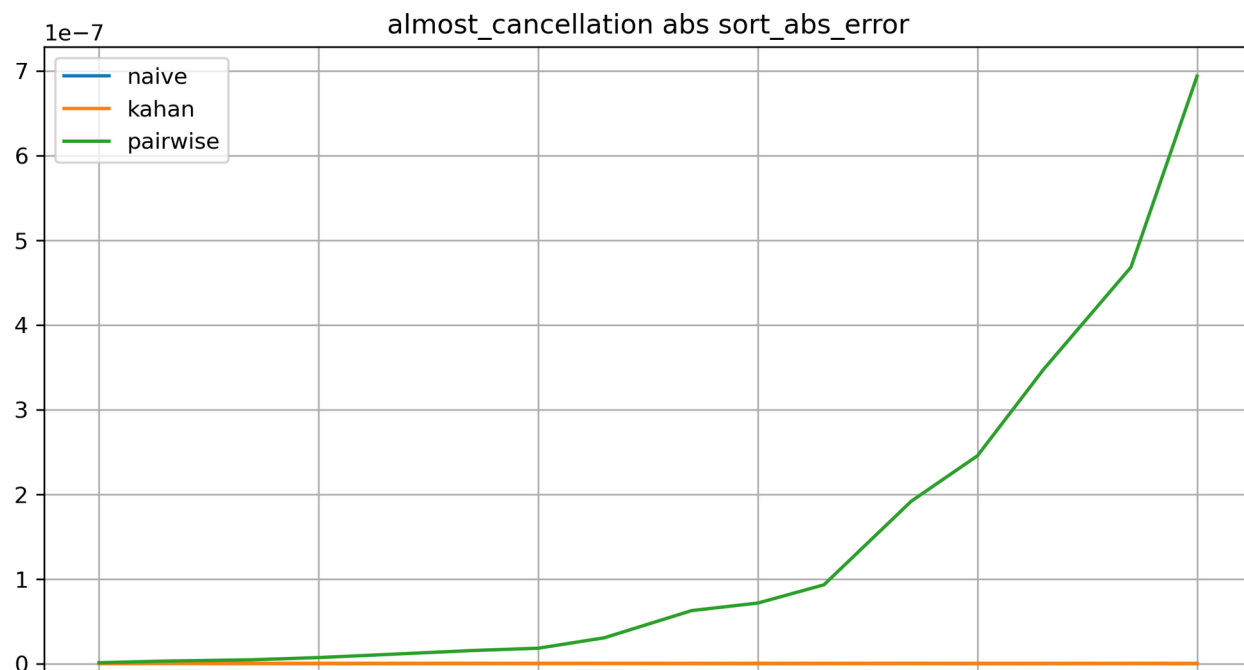


График 1

График 2

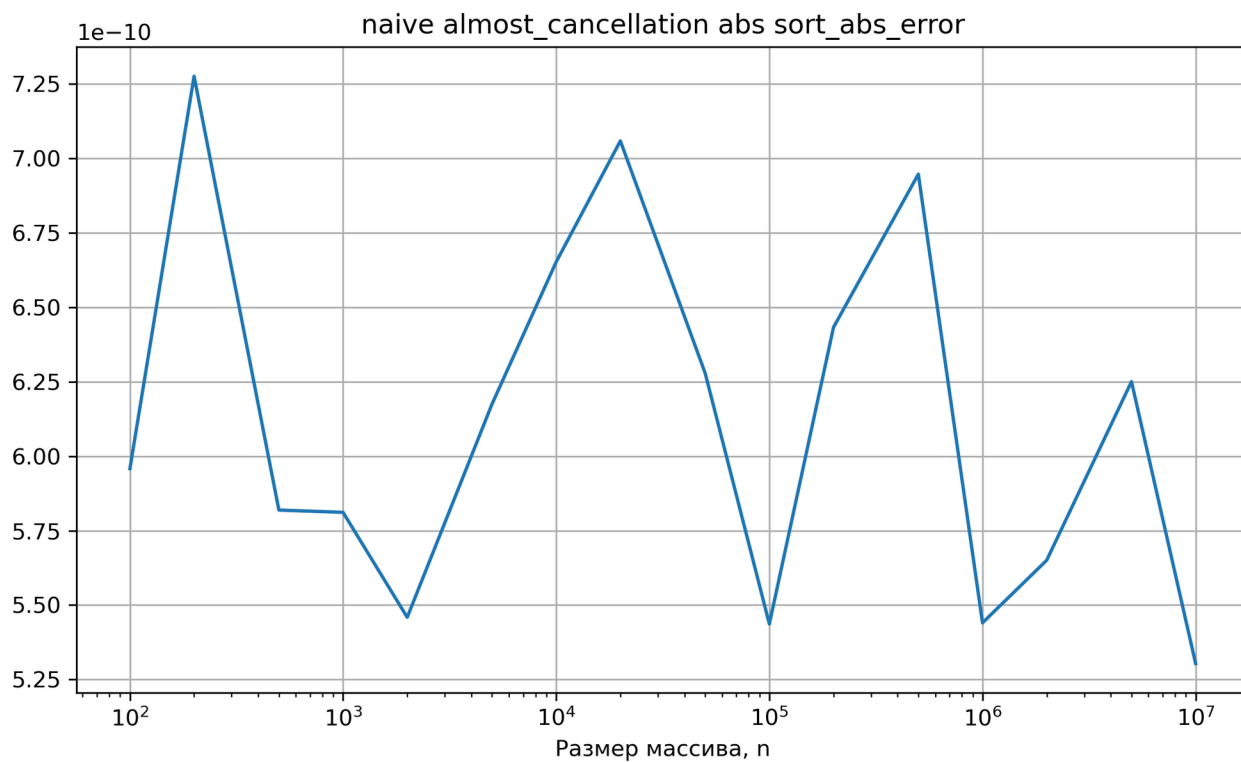


График 3

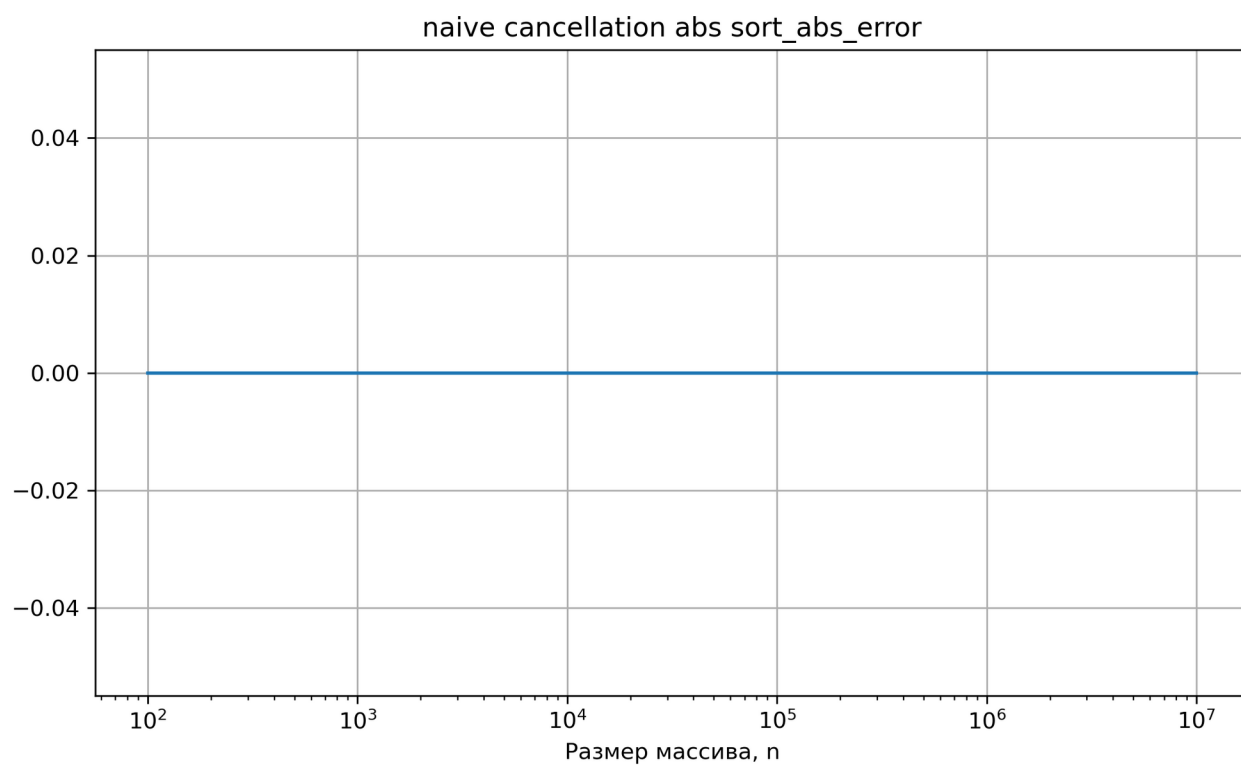


График 4

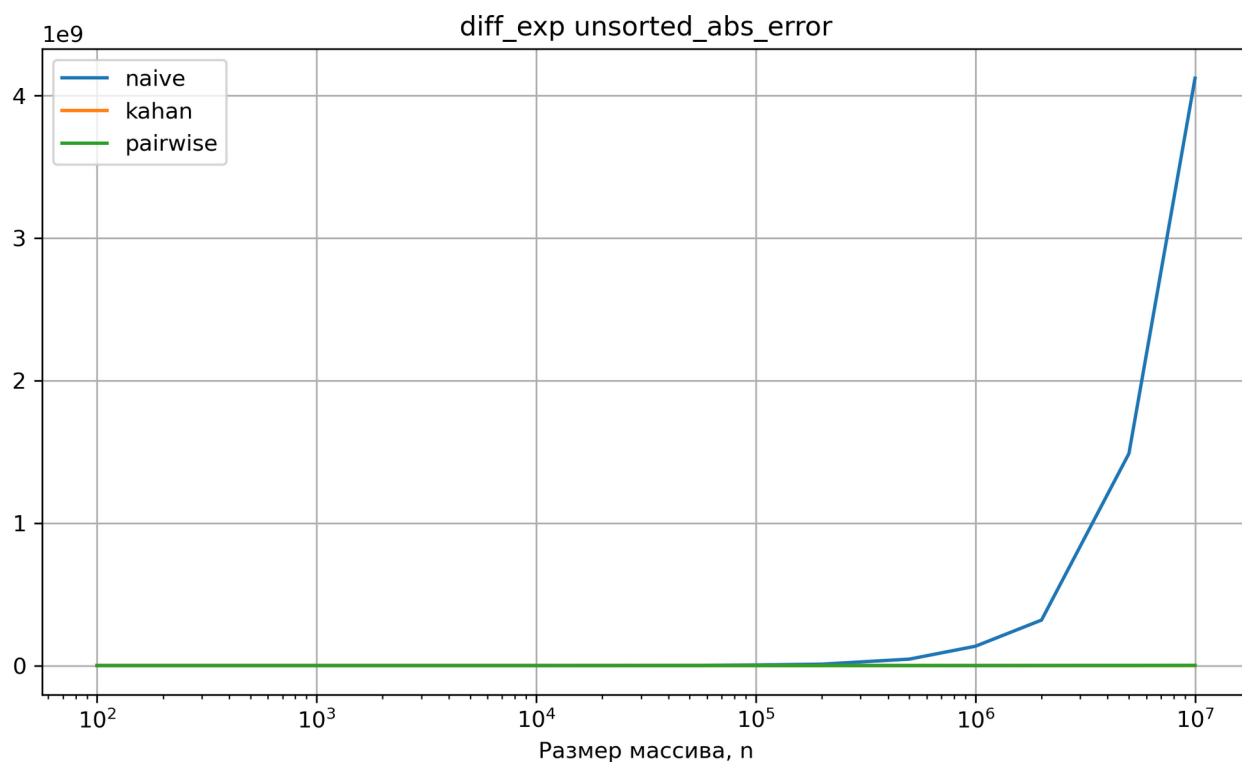


График 5

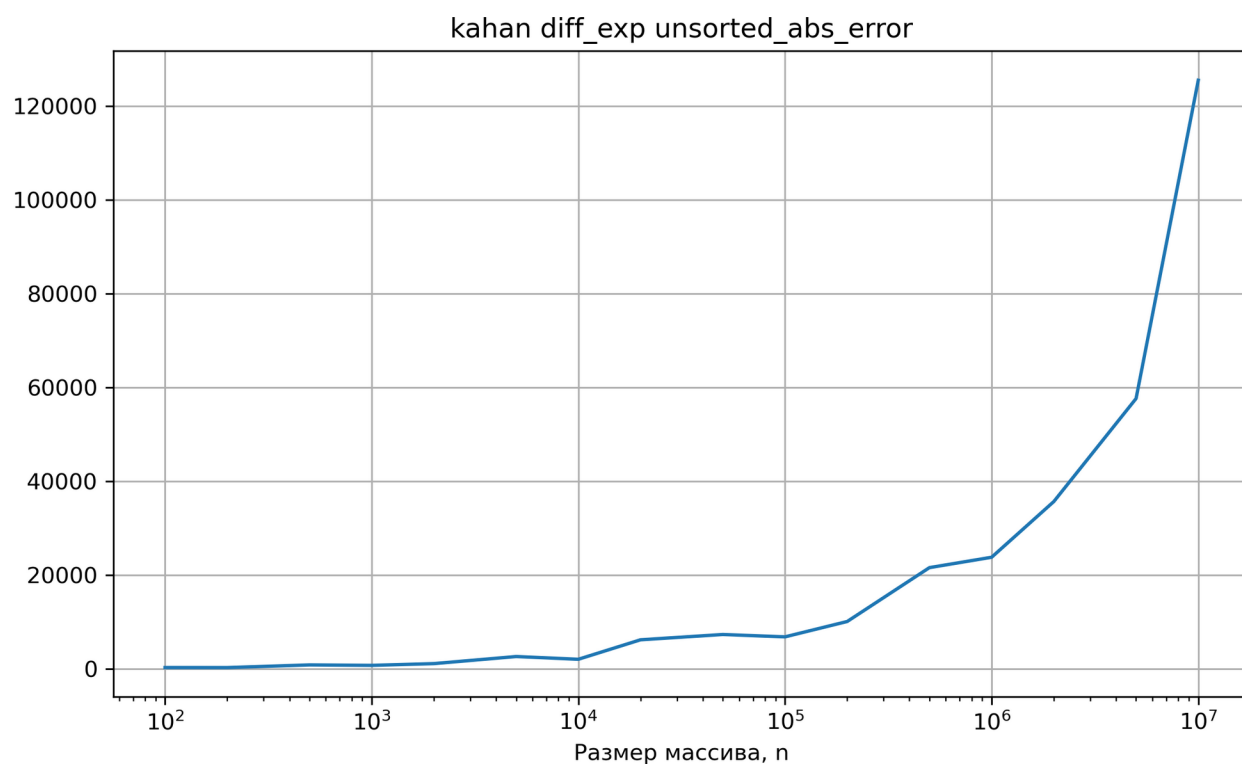


График 6

4.2 Относительная ошибка

Во всех типах тестов, где вычислялась относительная ошибка, наивное суммирование показало наибольшую относительную ошибку, кроме случая с почти сокращающимися парами, отсортированными по модулю (см. Графики 1, 2 в приложении 4.2). Это произошло

по той же причине, по которой наивное суммирование показало наименьшую абсолютную ошибку в пункте 4.1.

Второй график демонстрирует снижение относительной ошибки по мере увеличения массива. Это объясняется тем, что сумма маленького массива, у которого элементы сокращаются с шумом, очень мала.

Приложение 4.2:

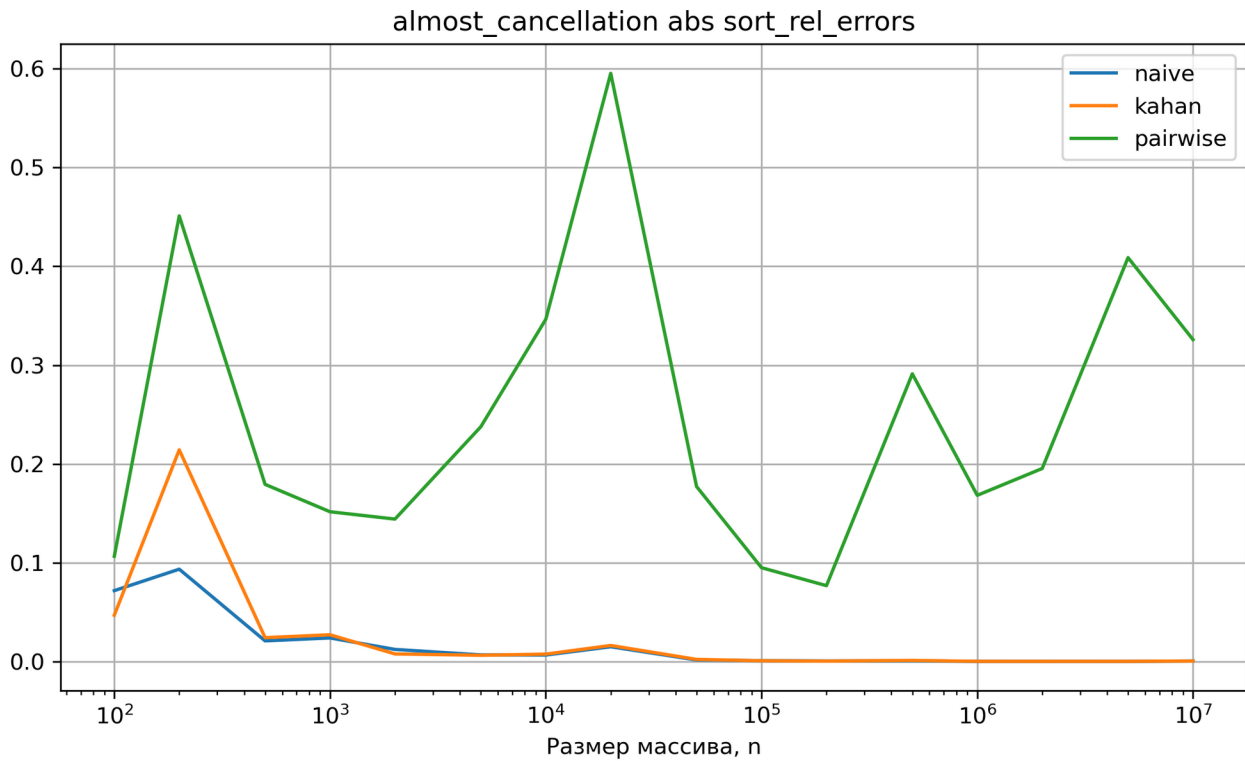


График 1

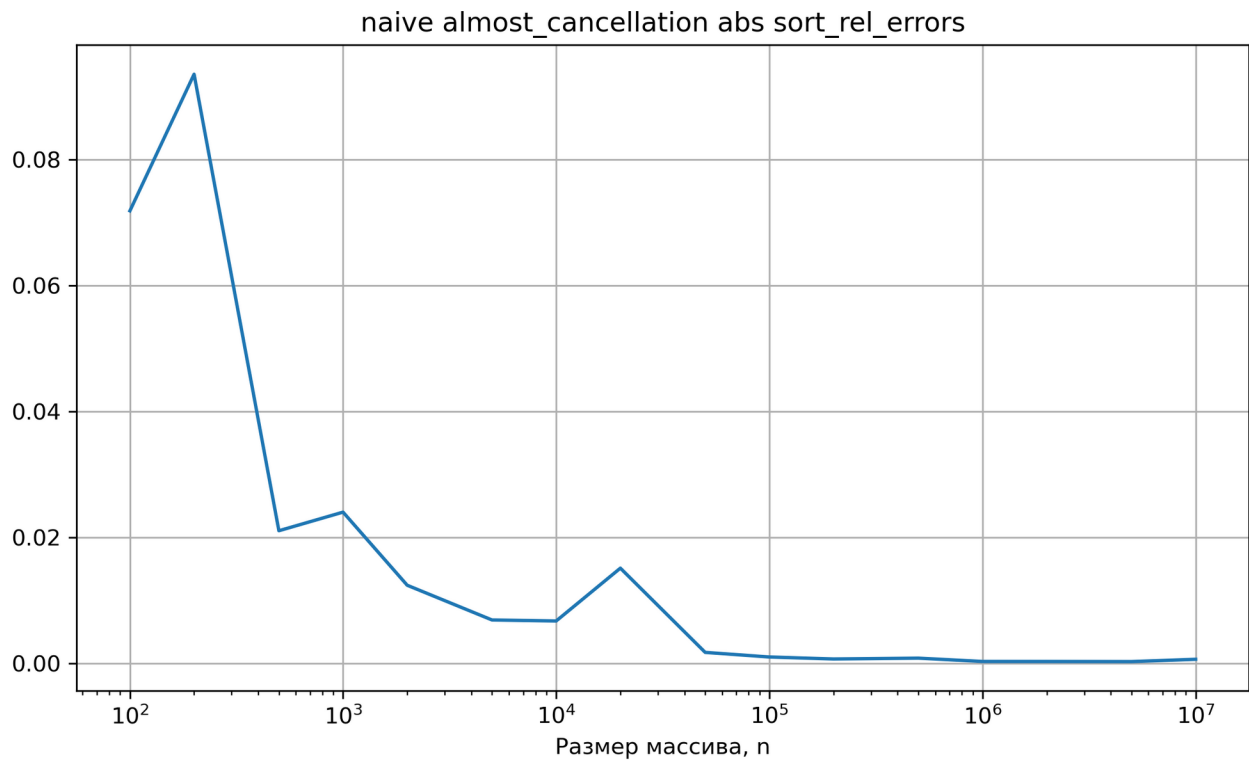


График 2

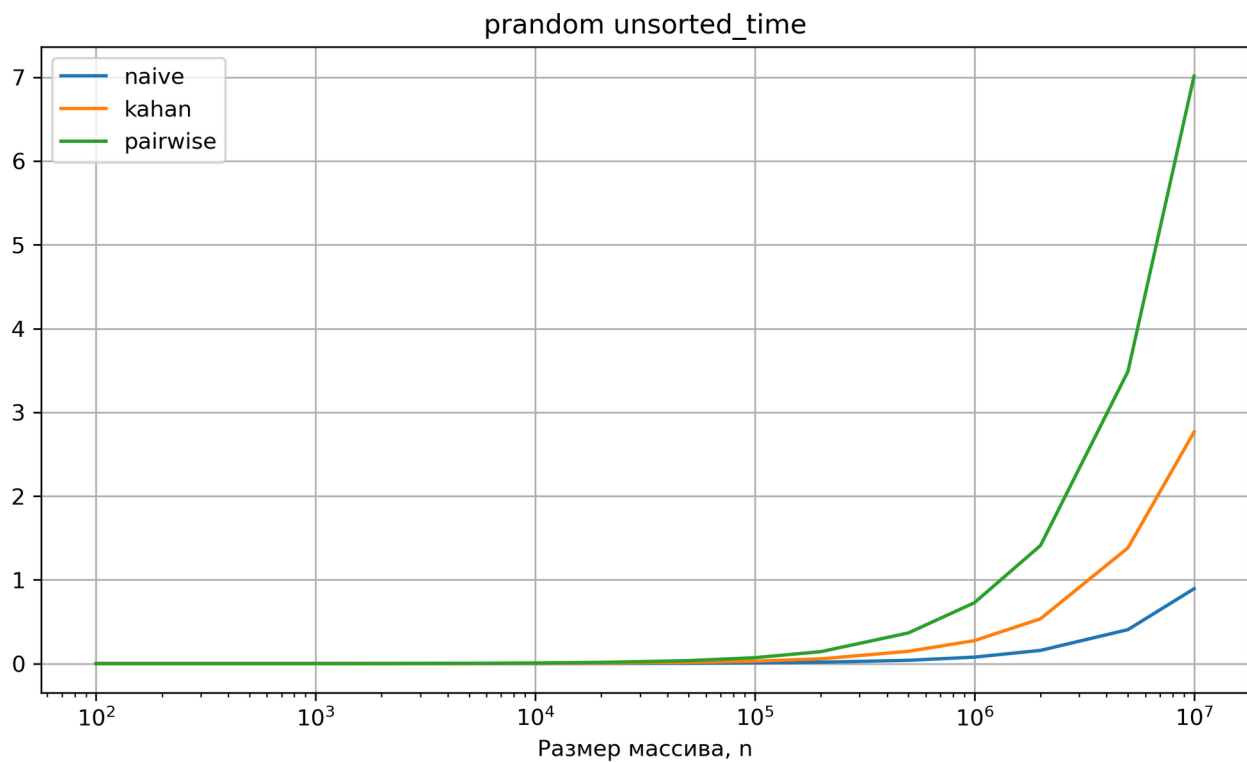
4.3 Время выполнения

При всех типах тестов получились очень похожие данные.

Наибольшее время выполнения было у попарного суммирования. Более низкая производительность, вероятно, связана с большим количеством рекурсивных вызовов и затратами на их организацию. Это является лишь гипотезой, так как время на вызов рекурсии не измерялось.

Наименьшее время выполнения во всех типах тестов у наивного суммирования. Это происходит потому, что алгоритм не делает сложных рекурсивных вызовов и делает всего две операции за итерацию вычисления суммы - сложение и присваивание, тогда как алгоритм Кэхэна делает больше операций на каждой итерации.

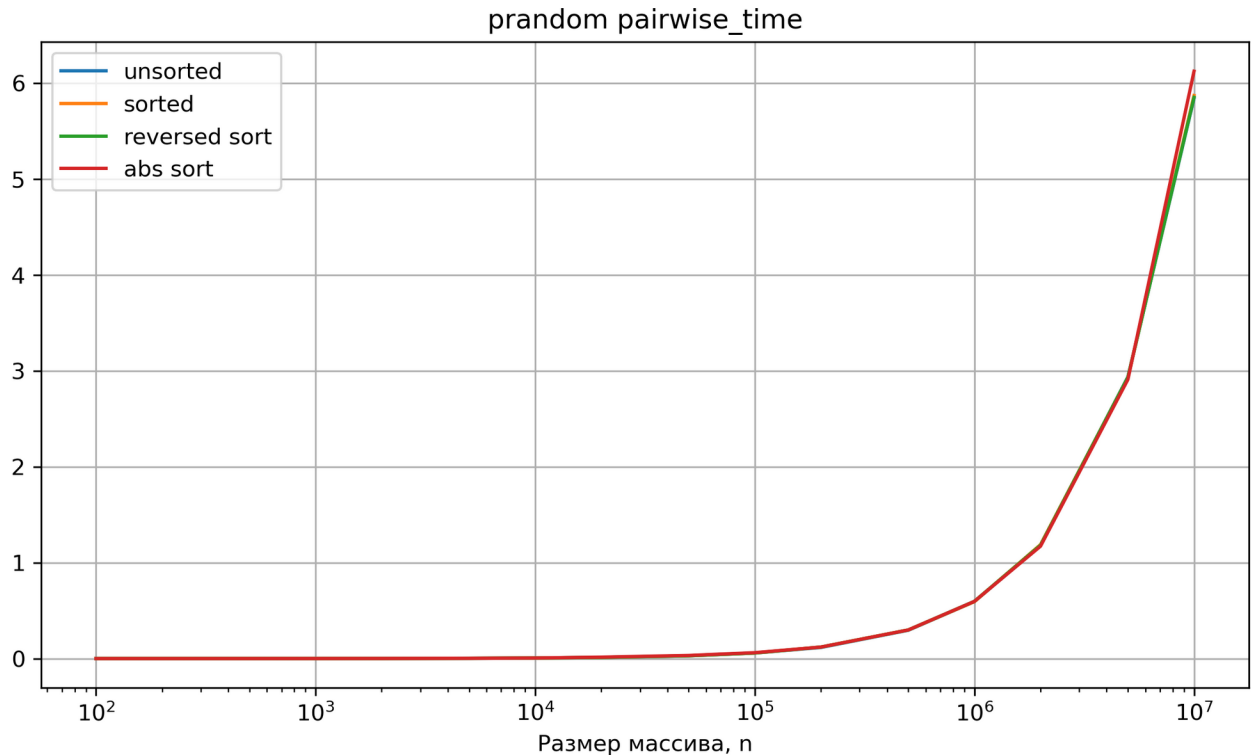
Приложение 4.3:



4.4 Влияние порядка элементов массива на время выполнения

Порядок элементов массива не влиял значимым образом на время выполнения всех алгоритмов. Это свидетельствует о том, что время выполнения зависит только от количества элементов в массиве.

Приложение 4.4:



4 . 5 Влияние порядка элементов массива на отн. ошибку

Относительную ошибку алгоритма Кэхэна почти во всех случаях снижала как сортировка по неубыванию, так и по невозрастанию. На большинстве исследованных датасетов, алгоритм Кэхэна показывал наибольшую относительную ошибку, кроме случая с гармоническими числами. Там наибольшую относительную ошибку возникла при сортировка по модулю и сортировка по неубыванию - они совпадают на графике. Для снижения относительной ошибки следует сортировать входные данные по невозрастанию.

Наивное суммирование показывало наихудший результат, когда массив был отсортирован по неубыванию и невозрастанию и лучший результат, если массив отсортирован по модулю. Таким образом, сортировка по модулю является более предпочтительной для алгоритма наивного суммирования с точки зрения уменьшения относительной ошибки.

Алгоритм попарного суммирования не показывает значимых отличий в относительной ошибке до размера массива 10^5 . Если размер массива больше, то наихудшим образом показывает себя в случаях, где массив отсортирован по неубыванию и невозрастанию. Наименьшая относительная ошибка возникала во всех случаях, когда массив был неотсортирован.

4 . 6 Влияние порядка элементов массива на абсолютную ошибку

В случае, когда алгоритм Кэхэна суммировал случайные вещественные числа $\in [-1;1]$, порядок элементов массива не влиял на абсолютную ошибку. Она всегда была равна нулю.

Во всех остальных типах датасетов, наибольшая относительная ошибка возникала на неотсортированных массивах, кроме случая с гармоническими числами. В том случае особенно плохо показывал себя на отсортированных по неубыванию и модулю массивах. В остальных случаях сортировка по модулю, по невозрастанию и по неубыванию не показывали значимых различий между собой. Таким образом, для алгоритма Кэхэна универсальной будет сортировка по невозрастанию.

Алгоритм наивного суммирования показывал наибольшую абсолютную ошибку на всех типах датасетов при сортировке по невозрастанию. Наименьшая абсолютная ошибка выходила при сортировке по модулю.

В случаях, где данные были отсортированы по невозрастанию алгоритм попарного суммирования показывал наибольшую абсолютную ошибку. Почти всегда, наименьшая абсолютная ошибка возникала, когда массив был не отсортирован.

5 . Выводы

Наивное суммирование можно использовать в областях, где скорость вычислений является главным критерием, а небольшие ошибки округления несущественны. Например, суммирование малого количества чисел.

Алгоритм Кэхэна подойдет в ситуациях, где требуется высокая точность вычислений, особенно при суммировании большого количества чисел, или если числа отличаются друг от друга на порядки. Подобные случаи возникают при статистической обработке данных или в научных вычислениях.

Попарное суммирование может представлять интерес в случаях, где возможно параллельное вычисление, ведь пары могут вычисляться независимо друг от друга. Я проводил исследование рекурсивной реализации алгоритма, поэтому эти преимущества не были продемонстрированы.

5 . 1 Основные выводы

- Наивный алгоритм суммирования показал наилучший результат по времени, но и наименьшую точность.
- Алгоритм Кэхэна обеспечивал наилучшую точность в большинстве экспериментов.
- Попарное суммирование оказалось средним по точности, но оказалось самым медленным, вероятно, из-за своей рекурсивной реализации.
- Порядок элементов не влиял на время существенным образом, но мог сильно увеличить точность.

5 . 2 Практические рекомендации

- Использовать наивное суммирование там, где точность несущественна, и важна лишь скорость выполнения.
- Уменьшить ошибку наивного суммирования помогает сортировка по модулю.
- Использовать алгоритм Кэхэна там, где первостепенна точность.

Проведенное исследование показывает, что даже такая простая операция, как суммирование чисел с плавающей точкой, обладает рядом особенностей, существенно влияющих на точность вычислений. Это следует учитывать при работе с числами с плавающей точкой.